

IMPLEMENTING SIGNAL PROCESSING ALGORITHMS FOR RADAR MEASUREMENTS USING LAUNCHXL-F28379D

GUIDE
Dr. Dhanesh G. Kurup

AUTHORS
Dheeraj Reddy B - BL.EN.U4ECE23112
Vaibhavi C R - BL.EN.U4ECE23152

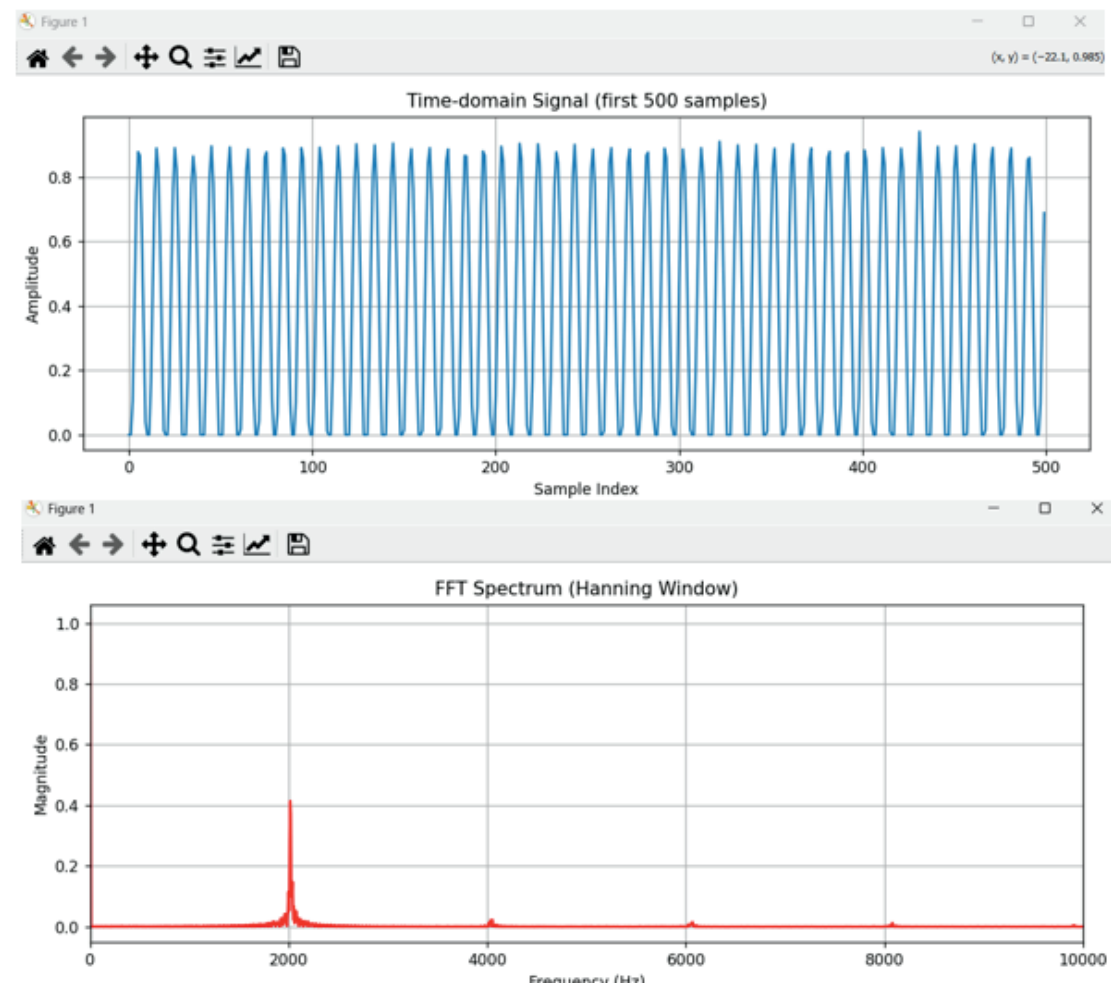
INTRODUCTION

Real-time signal processing is computationally intensive, especially in radar, autonomous vehicles, and wireless systems that demand fast and accurate extraction of spatial and velocity information. This project focuses on implementing Discrete Fourier Transform (DFT) and Fast Fourier Transform (FFT) algorithms on the Texas Instruments LAUNCHXL-F28379D DSP board.

The goal is to achieve efficient, real-time frequency-domain analysis optimized for high-speed computation, low memory usage, and accurate spectral estimation for radar and wireless applications.

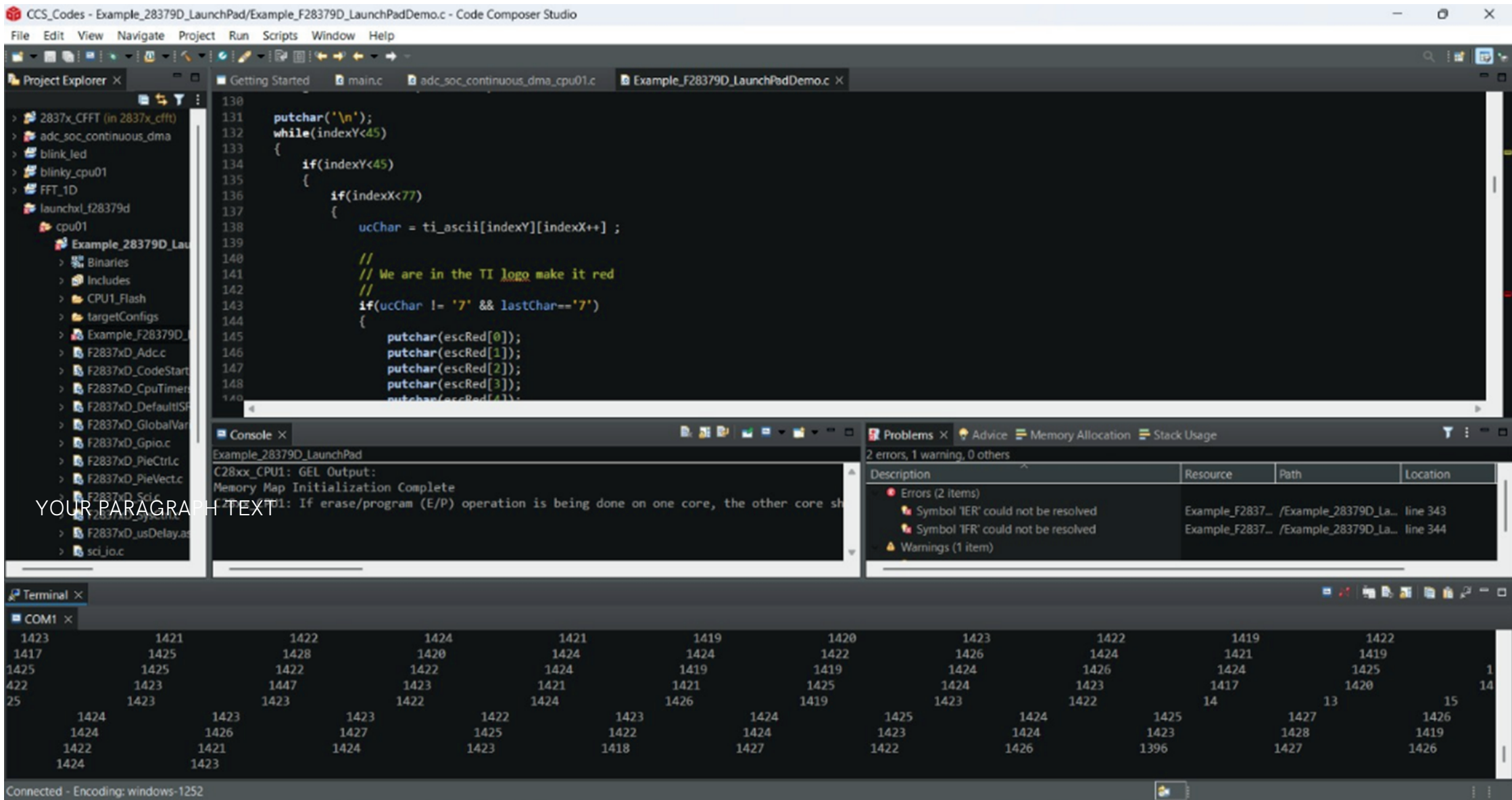
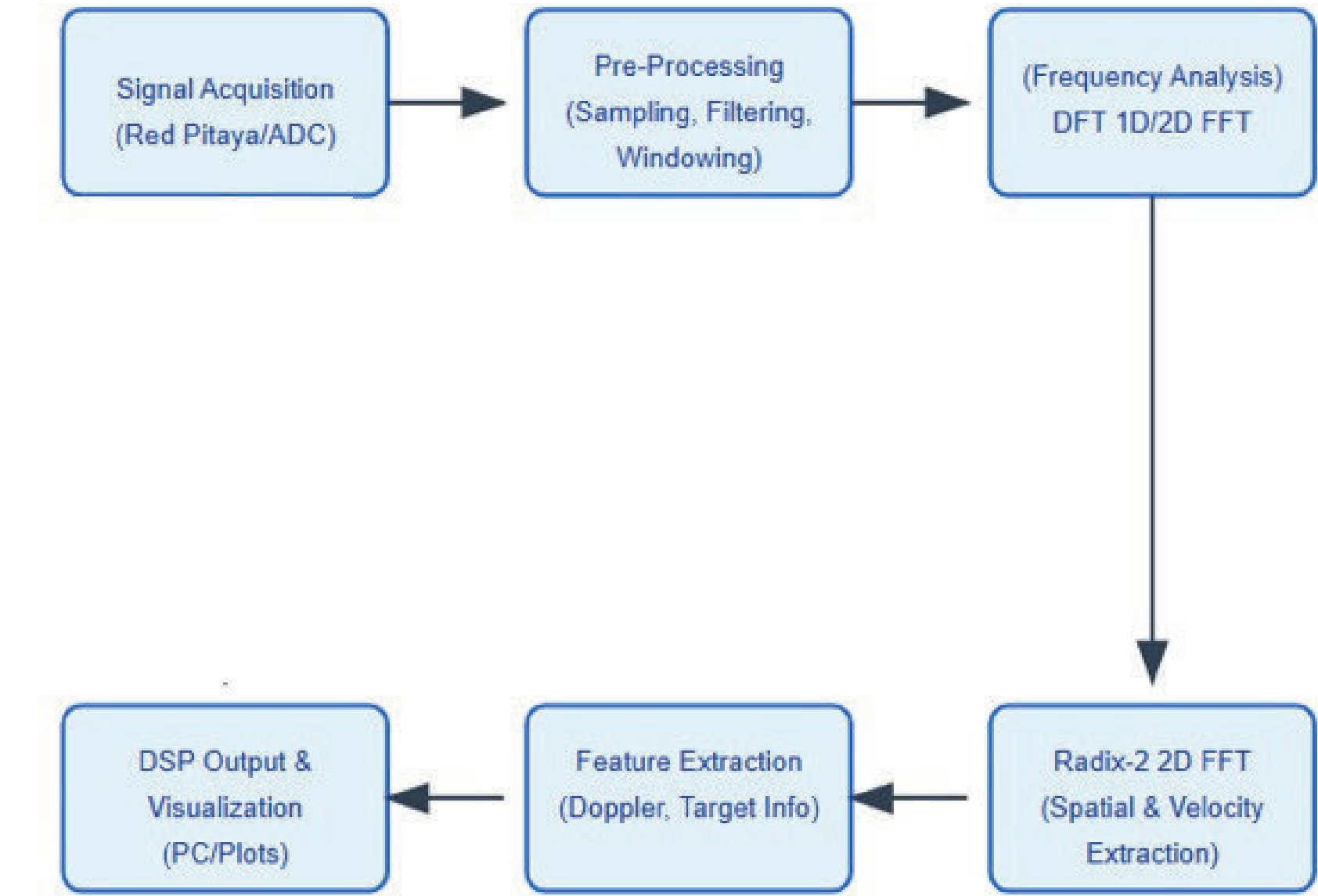
IMPLEMENTATION DETAILS

- Hardware: TI LAUNCHXL-F28379D DSP, ADC module for input signal sampling.
 - Software: C, Code Composer Studio, MATLAB, Python (for verification).
 - Libraries: KISS FFT Package and Arduino FFT Library for cross-platform testing.
 - Testing Conditions:
 - Frequencies: 2 kHz and 4 kHz test signals.
 - Sampling via loopback and real-time inputs.
- Optimization: Code adapted for on-chip parallel CPUs (CPU1 & CPU2). DMA used for efficient data transfer. make this para like



RESULTS/FINDINGS

- Successfully detected and analyzed frequency components of 2 kHz and 4 kHz signals.
 - KISS FFT and Arduino FFT implementations validated output consistency.
 - The DSP achieved real-time frequency-domain conversion with minimal latency.
- The system demonstrates feasibility for radar Doppler velocity extraction and wireless signal monitoring.



METHODOLOGY

- Signal Simulation
Generate tunable sine and noise-added signals using MATLAB/Python/C for algorithm validation.
- Algorithm Development
Implement Radix-2 DFT and FFT algorithms; extend to 2D FFT; verify accuracy of frequency detection.
- Porting to DSP
Optimize C-based FFT code for real-time execution on LAUNCHXL-F28379D. Utilize DSP libraries and efficient memory allocation.
- Testing & Validation
Perform loopback and signal injection tests. Analyze frequency peaks, velocity estimation, and spatial mapping.
- Final Integration
Integrate signal capture, processing, and visualization. Display results using MATLAB/Python plots.

DATA SHEET

- FFT Processing Capacity
- Efficient for up to 2048-point FFT in real time.
 - Higher FFT sizes (8k–16k) reduce throughput compared to high-end DSPs (like TI C6000).
- On-Chip RAM
- Around 200 KB total RAM (shared between program and data memory).
 - Can store about 2000–3000 samples per channel for 16-bit data.
 - Equivalent to roughly one 2k-point FFT frame.
- ADC Input Range
- Accepts 0–3.3 V analog input.
 - Signal must be scaled properly using conditioning circuitry.
- Data Communication
- File I/O functions (e.g., fopen, printf) not natively supported.
 - Use UART/SCI for serial transfer.
 - Use USB (high-speed interface) for large data streaming.

- ADC Sampling Rate
- Theoretical maximum of 3.5 MSPS.
 - Due to processing overhead, effective bandwidth $\approx$ 1–1.5 MHz in continuous acquisition.

FUTURE WORK

- Extend the current 1D FFT to a fully functional 2D FFT implementation.
- Utilize both CPUs (CPU1 & CPU2) for parallel FFT computation.
- Enhance ADC resolution and sampling rate.
- Optimize DMA for high-volume data transfer.
- Develop synchronization between CPUs for faster multi-channel processing.
- Benchmark computation time and accuracy improvements over existing versions.

CONCLUSION

The project demonstrates the real-time FFT capability of the TI LAUNCHXL-F28379D DSP for radar signal analysis. It validates the use of DSP-based embedded systems for efficient signal processing, paving the way for radar, sonar, and wireless sensing applications